Latches and Flip-Flops

Laboratory Assignment 9
ECE 201: Digital Circuits and Systems

Learning Objectives

After completing this lab, you will be able to

- explain the fundamental logic circuits used in construction of computer memories
- implement sequential logic circuits using VHDL

Equipment and Materials

- DE10-Lite Board
- Intel Quartus Prime Design Software v15.1 or v18.1

Pre-Lab Assignment

Prior to attending your assigned laboratory section, complete the following tasks

- Watch the first half of the following video that explains how basic computer memory elements are constructed using logic gates: Registers and RAM. Please note, for this assignment you only need to watch up until the 5:20 mark.
- Watch this video that briefly reviews two's complement binary numbers Binary and Two's Complement
- Watch this video that provides context on why two's complement is often used to represent signed numbers in computers Binary: Plusses & Minuses

Part I: D Flip-Flop

Perform the following:

1. In a new Quartus project, create a new top-level VHDL file that implements a D flip-flop using **behavioral VHDL**.
2. Include an **asynchronous, active-low reset** for your D flip-flop.
3. Run analysis and synthesis and correct any errors.
4. View the implementation in the RTL viewer and Technology Map Viewer and include screenshots in your report.
5. Simulate the D flip-flop in ModelSim and confirm that it functions as expected as you cycle the Clock signal.
6. Make the required pin assignments using the Pin Planner to implement the D flip-flop on your DE10 board:
   (a) Use switch SW_0 to drive the D input of the flip-flop.
   (b) Use button KEY_0 as a manual clock input. **Use the provided debounce.vhd and instructions to debounce the button signal for the clock input.** Use the pushbutton as the "button" input to the debounce component, use the "result" output as the clock input to the circuit, and connect the debounce.vhd "clk" to PIN_P11 for the 50 MHz FPGA clock.
   (c) Use button KEY_1 as the active-low Reset
   (d) Connect the Q output to $LEDR_0$.
7. Compile your circuit and program the board.
8. Test the functionality by toggling the D and *Clock* switches and observing the Q output.
9. **Demonstrate that your FPGA circuit functions correctly for values of D and *Clock* for the first part of the lab check-off.**

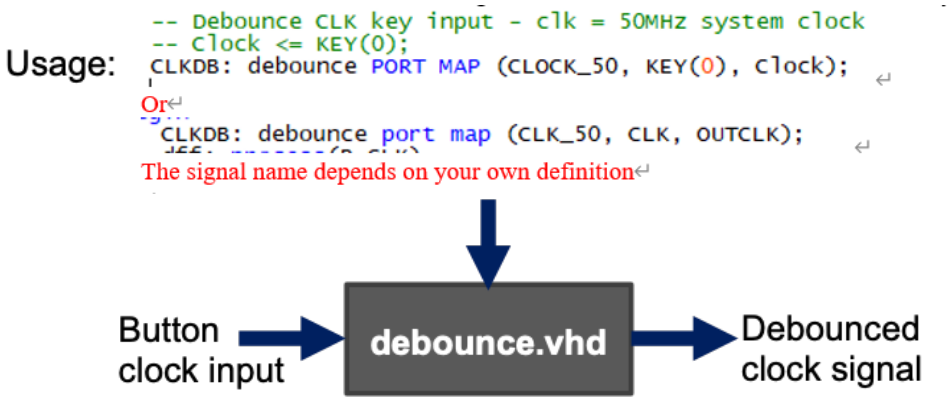

Figure 1: Illustration of debouncing using the provided component.

Part II: 8-Bit Accumulator Circuit

In this part, you will use components and port mapping to implement the circuit shown in Figure 2. This circuit, which is often called an *accumulator*, is used to add the value of an input A to itself repeatedly. Important parts of this circuit are described below.

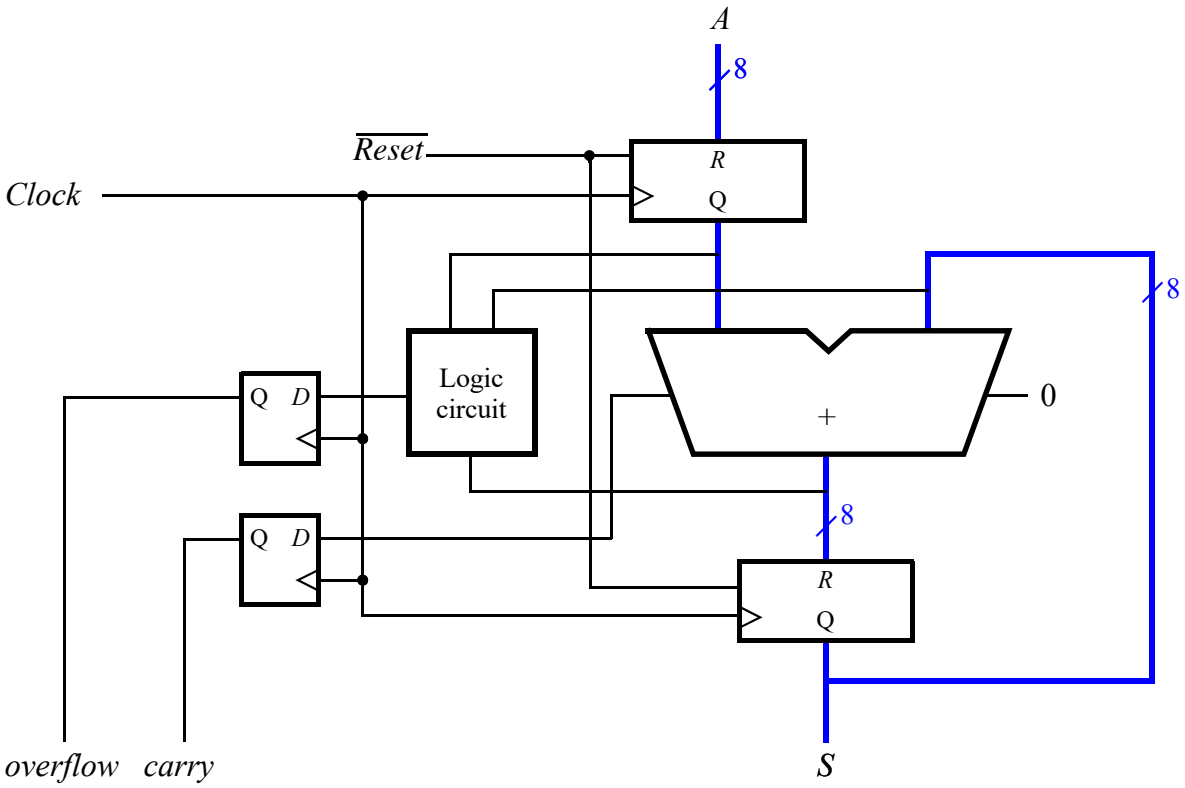

Figure 2: An 8-bit accumulator circuit.

1. The central component of the circuit is an 8-bit adder. Note the carry-out signal and the fixed 0 carry-in. You should use what you have from the previous lab for this adder component.

2. The two components labeled with R on the input and output of the adder are *registers*. You will implement this 8-bit register component using 8 parallel D flip-flops. The clock signal of each register is connected to all of the D flip-flops so that all 8 bits are "loaded" simultaneously on a clock edge, causing the input value to appear on the output of the register.

3. The *overflow* signal should indicate when the output of the addition is not valid, and can be determined from the inputs and outputs of the adder, with a design that you will implement for the component labeled "Logic circuit". If the input A is a 2's-complement number, then *overflow* should be set to 1 in the case where the adder output does not represent a correct 2's-complement result (Hint: Consider how you can detect an incorrect sign of the sum given the signs of each input value. There are two overflow cases that should be detected).

4. Two D flip-flops on the left store the values of *overflow* generated by your logic circuit and *carry* generated by the 8-bit adder. Use your D flip-flop component from the previous part.

Perform the following steps to implement this accumulator circuit. **You are encouraged to utilize components from previous labs and previous parts of this lab, the VHDL '+' operator for arithmetic, and statements like when/select in your architecture.** Test each component individually before connecting the full circuit.

1. Create a new Quartus project. Create a new VHDL file for the 8-bit adder shown in Figure 2, and add your code from the previous lab for the 8-bit adder. Make the adder your top-level entity and simulate to ensure that it functions properly before moving on to constructing the full circuit.

2. Create a new file and write VHDL code for an 8-bit register, utilizing 8 D flip-flop components, and include a Reset input signal for the register that is connected to the Reset signal of each D flip-flop component. Verify that the register functions correctly by itself by setting it as your top-level entity and simulating (remember to test the Reset signal).

3. Create a new VHDL file and write VHDL code for your overflow logic circuit. The overflow circuit output should be 1 if the sign of the sum is not consistent with the sign of the addends. Verify that the circuit functions correctly by itself by setting it as your top-level entity and simulating.

4. Create a new VHDL file for the full accumulator circuit in Figure 2, and make this your new top-level entity. Connect the components using port maps. Use two instances of your D flip-flop to store the overflow and carry, and use two instances of your register on the input and output of the adder. Connect one Reset input signal to both registers.

5. Use simulation to verify the correct operation of the complete accumulator circuit. Note: To initialize the S output, you will need to have Reset at 0 for a short time at the beginning of the simulation, then set Reset to 1 for the remainder of the simulation.
   （When simulating the code of the top-level entity, do not declare debounce related components, and do not instantiate and port map debounce, otherwise the simulation will not be correct.
   After the simulation is complete, you can add the debounce function）

6. You will be displaying the register output values as hexadecimal numbers on the 7-segment displays, so add your hexadecimal to 7-segment decoder file to the project and update your accumulator output connections accordingly (or, create a new top-level entity to connect your accumulator signals to the decoders).

7. Make the necessary pin assignments needed to implement the circuit on your DE-10 board:

   (a) Connect input A to switches SW_{7-0}

   (b) Connect the active-low Reset to SW_9

   (c) The sum from the adder should be displayed on the red lights $LEDR_{7-0}$

   (d) The registered *carry* signal should be displayed on $LEDR_8$

   (e) The registered *over low* signal should be displayed on $LEDR_9$

   (f) Show the registered values of *A* and *S* as hexadecimal numbers on the 7-segment displays HEX$3-2$ and HEX$1-0$, respectively

   (g) Use KEY_0 as a manual clock input. **Use the provided debounce.vhd and instructions to debounce the button signal for the clock input.** Use the pushbutton as the "button" input to the debounce component, use the "result" output as the clock input to the circuit, and connect the debounce.vhd "clk" to PIN_P11 for the 50 MHz FPGA clock.

8. Compile the circuit, download it onto your DE10 board, and test it by using different values of A. Be sure to check that the *over low* output works correctly.

9. **Demonstrate that your circuit functions as an accumulator for various values of A for the second part of the lab checkoff.**